Project Implementation Plan

Objective: Machine Learning for Antimicrobial Resistance (AMR) Prediction from Bacterial Genomes **Organism:** *Klebsiella pneumoniae* **Antibiotic Target:** Meropenem (Carbapenem class) **Task:** Binary Classification (Susceptible vs. Resistant)

Phase 1: Data Acquisition & Quality Control (QC)

Goal: Establish a clean, reliable biological ground truth before doing any math.

1. **The Labels (Phenotype):**
   - Load the BV-BRC metadata CSV into Pandas.
   - Filter the `Resistant Phenotype` column to strictly retain "Susceptible" and "Resistant" (or "Non-Susceptible" if you've confirmed it strictly means resistant).
   - **Drop** any "Intermediate" labels to ensure the model learns definitive resistance.
   - Map labels to a binary vector (y): `Susceptible = 0`, `Resistant = 1`.
2. **The Genomes (Genotype):**
   - You have downloaded ~4,328 Multi-FASTA (`.fna`) files containing Whole Genome Shotgun (WGS) contigs.
   - **Run Bash QC Checks:**
     - *File Size:* `find . -name "*.fna" -size -1M -ls` (Delete files < 1MB).
     - *Header Integrity:* `awk 'FNR==1 && /^[^>]/ {print FILENAME}' *.fna` (Delete files that don't start with `>`).
     - *Assembly Quality:* `grep -c "^>" *.fna | sort -t: -k2 -nr | head -n 20` (Delete highly fragmented genomes with >500 contigs).
   - **Sync:** Ensure any `.fna` files deleted during QC are also dropped from your Pandas DataFrame so X and y align perfectly.

Phase 2: Preventing Clonal Data Leakage

Goal: Group genetically identical/highly similar bacteria so they do not bleed across the train/test split.

1. **Genome Sketching:**
   - Instead of CD-HIT (which is too slow for 4,300 whole genomes), use **Mash** (MinHash algorithm).
   - Run Mash to compute pairwise distances between all 4,300 genomes.
2. **Clustering:**
   - Cluster the genomes at a ~95% Average Nucleotide Identity (ANI) threshold.
   - Assign a `Cluster_ID` to every genome in your metadata CSV.
3. **Grouped Splitting:**

- Use `GroupShuffleSplit` (from `scikit-learn`) based on the `Cluster_ID`.
- This guarantees that if Hospital A had an outbreak of 50 identical *K. pneumoniae* clones, all 50 go entirely into the training set, or entirely into the testing set. The model must learn to generalize.

Phase 3: Feature Engineering (K-mer Extraction)

Goal: Tokenize DNA text into a numerical feature matrix safely.

1. **Automated KMC Extraction:**
   - Write a Bash script to loop through all `.fna` files and extract 31-mers.
   - *Example loop logic:*

```
mkdir -p kmer_counts tmp
for file in *.fna; do
    base=$(basename $file .fna)
    # Extract 31-mers (-k31), FASTA input (-fm), min count 1 (-ci1)
    kmc -k31 -fm -ci1 $file tmp_db/$base tmp/
    # Dump to readable text format
    kmc_tools transform tmp_db/$base dump kmer_counts/${base}.txt
done
```

2. **Memory-Safe Matrix Assembly:**
   - You are building a matrix of ~4,300 rows and ~10,000,000 columns.
   - **DO NOT** use standard Pandas DataFrames or NumPy arrays; your RAM will crash.
   - Initialize a `scipy.sparse.coo_matrix` or `dok_matrix`.
   - Parse the text files in Python line-by-line, adding counts to the sparse matrix.
   - Convert the final assembled matrix to a `csr_matrix` (Compressed Sparse Row) for ultra-efficient ML training.

Phase 4: Model Training & Evaluation

Goal: Train a tree-based classifier to handle extreme high-dimensionality and sparsity.

1. **Algorithm Selection:**
   - Use **XGBoost** or **LightGBM**. LightGBM is highly recommended here as it is specifically optimized for sparse, high-dimensional tabular data and uses less memory.
2. **Handling Clinical Imbalance:**
   - Your dataset is likely skewed (e.g., more susceptible cases than resistant).
   - Calculate the imbalance ratio and pass it to the `scale_pos_weight` parameter in XGBoost/LightGBM. This penalizes the model heavily for missing resistant cases (False Negatives).
3. **Rigorous Metrics:**

- **Discard Accuracy.**
- Optimize and evaluate using **Macro F1-Score** (balances precision and recall across both classes) and **MCC (Matthews Correlation Coefficient)** (the gold standard for imbalanced binary classification).

Phase 5: Biological Interpretability (The "Black-Box" Fix)

Goal: Prove the mathematical model reverse-engineered real-world molecular biology.

1. **SHAP Analysis:**
   - Pass your trained tree model to `shap.TreeExplainer`.
   - Extract the top 20 most globally important features (which represent 31-mer DNA strings).
2. **BLAST Validation:**
   - Take those top 20 k-mer strings.
   - Stitch together any overlapping k-mers into slightly longer sequences ("unitigs") if necessary.
   - Search them using BLAST against the **CARD (Comprehensive Antibiotic Resistance Database)**.
3. **The Expected Result:**
   - Your top features should perfectly map to known Carbapenemase genes, such as *blaKPC*, *blaNDM*, or *blaOXA-48*.
   - This proves your AI successfully identified the exact genetic mechanisms causing Meropenem resistance without any prior biological instructions.